

Remote Development Project Report

Student Name: [Your Name]

Student ID: [Your Student ID]

System Configuration

This project was completed in a Linux development environment running inside Windows Subsystem for Linux (WSL2). WSL was used for command-line operations, Python program development, testing, and Markdown documentation. The final PDF version was exported from the Windows host using a headless browser command after the Markdown report was completed.

Item	Information
Development Environment	Windows + WSL2
Linux Distribution	Ubuntu 24.04.4 LTS (Noble Numbat)
CPU Model	Intel(R) Core(TM) Ultra 9 285H
Memory Size	15 GiB
Operating System Version	Linux localhost 6.6.87.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 5 18:30:46 UTC 2025 x86_64 x86_64 x86_64 GNU/Linux
Compiler Version	GCC not installed in current WSL environment (gcc: command not found)
Python Version	Python 3.12.3

Command Output Evidence

CPU Model

Architecture:	x86_64
CPU(s):	16
Vendor ID:	GenuineIntel
Model name:	Intel(R) Core(TM) Ultra 9 285H
Thread(s) per core:	1

Core(s) per socket:	16
Socket(s):	1

Memory Size

	total	used	free	shared	buff/cache	available
Mem:	15Gi	618Mi	14Gi	3.6Mi	388Mi	14Gi
Swap:	4.0Gi	0B	4.0Gi			

Operating System Version

Linux localhost 6.6.87.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 5 18:30:46 UTC 2025 x86_64 x86_64 x86_64 GNU/Linux
--

Compiler Version

bash: line 1: gcc: command not found

Python Version

Python 3.12.3

Implementation Details

The goal of this project was to practice Unix/Linux command-line operations, write technical documentation in Markdown, and implement matrix multiplication in an interpreted language. All tasks were completed inside WSL, which provided a Linux-based workflow on top of a Windows host system.

Command-Line Workflow

Command	Purpose
mkdir -p ~/assignment1	Create the project directory
cd ~/assignment1	Enter the working directory
lscpu	Check CPU information
free -h	Check memory usage and size

<code>uname -a</code>	Check OS and kernel information
<code>gcc --version</code>	Check compiler availability
<code>python3 --version</code>	Check Python version
<code>python3 matrix_mul.py</code>	Run the Python matrix multiplication program
<code>python3 test_matrix_mul.py</code>	Run correctness tests
<code>msedge --headless=new --print-to-pdf=assignment1_report.pdf assignment1_report.html</code>	Export the final report to PDF

Documentation Workflow

Markdown was used for this report because it is simple, readable, and suitable for technical documentation. After the report was completed, a formatted HTML version was exported to PDF for submission.

Python Language Implementation

Source Code

The Python program below implements matrix multiplication using the standard triple-loop algorithm. It first checks whether the matrix dimensions are compatible and then calculates each element of the result matrix.

```
def matmul(A, B):
    rows_a = len(A)
    cols_a = len(A[0])
    rows_b = len(B)
    cols_b = len(B[0])

    if cols_a != rows_b:
        raise ValueError("Matrix dimensions do not match for multiplication.")

    result = [[0 for _ in range(cols_b)] for _ in range(rows_a)]

    for i in range(rows_a):
        for j in range(cols_b):
            for k in range(cols_a):
                result[i][j] += A[i][k] * B[k][j]
```

```
return result
```

Execution Command

```
python3 matrix_mul.py
```

Sample Output

```
Matrix A:
[1, 2]
[3, 4]

Matrix B:
[5, 6]
[7, 8]

A x B:
[19, 22]
[43, 50]
```

Algorithm Verification

To verify the correctness of the matrix multiplication algorithm, several test cases were used.

Verification Method

- 1. A 2x2 matrix example was checked against a manually computed result.
- 2. An identity matrix test confirmed that $A \times I = A$.
- 3. A zero matrix test confirmed that multiplying by a zero matrix produces a zero matrix.
- 4. A dimension mismatch test confirmed that invalid input raises a `ValueError`.

Test Cases

Test Case	Input Description	Expected Result	Actual Result
Small matrix test	2x2 matrix multiplied by 2x2 matrix	[[19, 22], [43, 50]]	Correct
Identity matrix test	$A \times I$	Result should equal A	Correct

Zero matrix test	$A \times \emptyset$	Result should be a zero matrix	Correct
Dimension mismatch test	Invalid dimensions	Program should raise ValueError	Correct

Test Script

```
from matrix_mul import matmul

def test_small_matrix():
    A = [[1, 2], [3, 4]]
    B = [[5, 6], [7, 8]]
    expected = [[19, 22], [43, 50]]
    assert matmul(A, B) == expected

def test_identity_matrix():
    A = [[2, 3], [4, 5]]
    I = [[1, 0], [0, 1]]
    assert matmul(A, I) == A

def test_zero_matrix():
    A = [[2, 3], [4, 5]]
    Z = [[0, 0], [0, 0]]
    expected = [[0, 0], [0, 0]]
    assert matmul(A, Z) == expected

def test_dimension_mismatch():
    A = [[1, 2, 3]]
    B = [[1, 2], [3, 4]]
    try:
        matmul(A, B)
        raise AssertionError("Expected ValueError was not raised")
    except ValueError:
        pass
```

Test Execution Command

```
python3 test_matrix_mul.py
```

Test Result

All tests passed.

C Language Implementation and Performance Analysis (Bonus)

This section is optional in the assignment description. In the current WSL environment, GCC was not installed at the time of this submission, so a compiled-language implementation was not included. If needed, this section can be extended later after installing the compiler toolchain in WSL.

Conclusion

This project helped me become more familiar with Unix/Linux command-line operations in a WSL environment and improved my ability to write technical reports in Markdown. I also implemented matrix multiplication in Python and verified its correctness with multiple test cases. Overall, the project provided practical experience in Linux-based development, algorithm implementation, and result validation.

References

1. Python Official Documentation
2. Ubuntu Documentation
3. Linux manual pages
4. Course materials and assignment instructions

Appendix

Additional Notes

- The project was developed in Windows Subsystem for Linux (WSL2).
- The matrix multiplication algorithm used in this project is the standard triple-loop method.
- For square matrices of size $n \times n$, the time complexity of this algorithm is $O(n^3)$.
- GCC was not installed in the current WSL environment at the time this report was prepared, so the compiled-language section was not included.